

the trick).



C++ Strings

C++ provides following two types of string representation

The C style Character string

The string class type introduced with standard C++

In C++, strings (text) can be represented in **two main ways**. These are the ones your slide mentions. Let's understand them clearly. 🧠💻

1 C-Style Character Strings

C-style strings come from the **C programming language**.

They are **arrays of characters** that end with a **null character** `\0`.

Example

```
</> C++  
  
#include <iostream>  
using namespace std;  
  
int main()  
{  
    char str[] = "Hello";  
  
    cout << str;  
  
    return 0;  
}
```



How it is stored in memory

H e l l o \0



`\0` marks the **end of the string**.

Important Points

- Stored as **character arrays**
- Ends with **null character** `\0`
- Uses library functions from `<cstring>`

Examples of functions:

- `strlen()` → find length
- `strcpy()` → copy strings
- `strcat()` → join strings
- `strcmp()` → compare strings

Example:

 C++ 

```
#include <iostream>
#include <cstring>
using namespace std;

int main()
{
    char str1[20] = "Hello";
    char str2[] = "World";

    strcat(str1, str2);

    cout << str1;

    return 0;
}
```

Output

HelloWorld 

2 C++ `string` Class

Modern C++ provides the `string` class in the `<string>` library.

It is **much easier and safer to use** than C-style strings.

Example

</> C++



```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string str = "Hello";

    cout << str;

    return 0;
}
```

Example: Joining Strings

</> C++



```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string a = "Hello ";
    string b = "World";

    string c = a + b;

    cout << c;

    return 0;
}
```

Output

Hello World





Comparison

Feature	C-Style String	C++ String Class
Type	Character array	<code>string</code> object
Library	<code><cstring></code>	<code><string></code>
Safety	Less safe	Safer
Ease of use	Harder	Easier



Easy Way to Remember

```
char str[] = "Hello";   → C-style string
string str = "Hello";   → C++ string
```



✅ In modern C++, programmers usually prefer the `string` class because it is simpler, safer, and more powerful.

If you want, I can also ↪ explain C++ string functions like `length()`, `append()`, `substr()`, `find()`, which are very common in exams.



C++ supports a wide range of functions that manipulate null-terminated Strings

In C++ (and C), null-terminated strings are C-style character strings that end with a null character `\0`.

C++ provides many functions to manipulate these strings, mainly through the `<cstring>` library. 🖥️

A null-terminated string example:

</> C++

```
char str[] = "Hello";
```



Memory representation:

H e l l o \0



The `\0` marks the end of the string.

◆ Common Functions for Null-Terminated Strings

These functions are found in the `<cstring>` library.

1 `strlen()` – Find Length of String

Returns the number of characters in a string (excluding `\0`).

Example:

</> C++

```
#include <iostream>
#include <cstring>
using namespace std;

int main()
{
    char str[] = "Hello";

    cout << strlen(str);

    return 0;
}
```



Output:

5



2 strcpy() – Copy String

Copies one string into another.

Example:

</> C++



```
#include <iostream>
#include <cstring>
using namespace std;

int main()
{
    char str1[20];
    char str2[] = "Hello";

    strcpy(str1, str2);

    cout << str1;

    return 0;
}
```

Output:

Hello



3 strcat() – Concatenate Strings

Joins two strings together.

Example:

</> C++



```
#include <iostream>
#include <cstring>
using namespace std;

int main()
{
    char str1[20] = "Hello ";
```

```
char str2[] = "World";

strcat(str1, str2);

cout << str1;

return 0;
}
```

Output:

Hello World



4 strcmp() – Compare Strings

Compares two strings.

Example:

```
<> C++

#include <iostream>
#include <cstring>
using namespace std;

int main()
{
    char str1[] = "Apple";
    char str2[] = "Apple";

    cout << strcmp(str1, str2);

    return 0;
}
```



Output:

0



Meaning:

Result

Meaning

0

Strings are equal

Result	Meaning
<0	First string is smaller
>0	First string is greater

Summary of Functions

Function	Purpose
<code>strlen()</code>	Finds string length
<code>strcpy()</code>	Copies string
<code>strcat()</code>	Joins two strings
<code>strcmp()</code>	Compares strings

Key Idea

C++ supports many functions to **manipulate null-terminated strings**, such as:

```
strlen
strcpy
strcat
strcmp
```



These functions are included in:

```
</> C++
```

```
#include <cstring>
```



 If you'd like, I can also explain the **difference between** `<cstring>` functions and C++ `string` class functions, which is a **very common exam question**.

